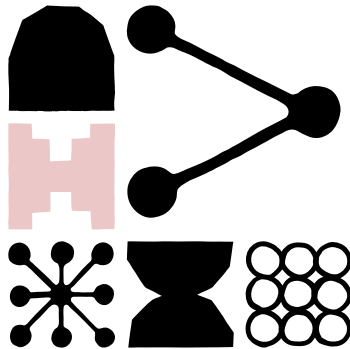


借助 Agent 为 Agent 编写高效工具

Writing effective tools for agents — with agents

Anthropic · 2025-09-11 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



Agent 的效能取决于我们为它提供的工具。本文将分享如何编写高质量工具和评测，以及如何让 Claude 为自己优化工具，从而提升表现。

模型上下文协议（Model Context Protocol, MCP）可以让大语言模型（LLM）Agent 使用多达数百种工具，解决现实世界中的任务。但我们怎样才能让这些工具发挥最大的效用？

本文介绍我们在多种 Agent 式 AI 系统中提升表现时最有效的一些方法¹。

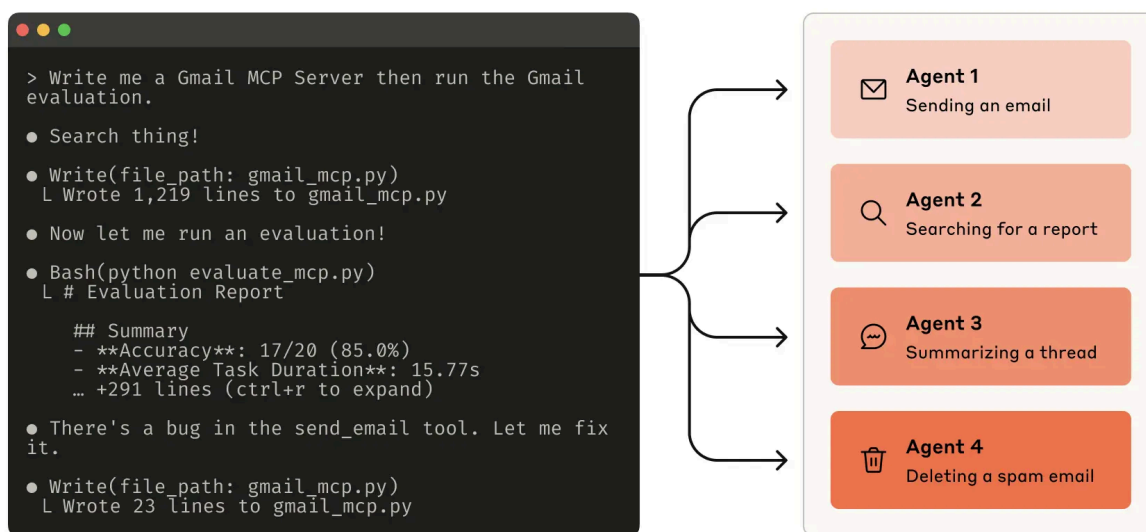
我们首先介绍如何：

- 构建并测试工具原型
- 创建并运行全面的评测，检验 Agent 使用工具的表现
- 与 Claude Code 等 Agent 协作，自动提升工具的表现

最后，我们总结在这一过程中发现的编写高质量工具的关键原则：

- 选择合适的工具进行实现，并决定哪些工具不应实现
- 通过命名空间为工具划定清晰的功能边界
- 让工具向 Agent 返回有意义的上下文
- 优化工具响应，提高 token 使用效率
- 对工具描述和规格说明进行提示词工程优化

Collaborating with Claude Code



Building an evaluation allows you to systematically measure the performance of your tools. You can use Claude Code to automatically optimize your tools against this evaluation.

什么是工具？

在计算领域，确定性系统在输入相同时，每次都会产生相同的输出；而 Agent 这样的非确定性系统，即使初始条件相同，也可能生成不同的响应。

在传统的软件开发中，我们是在确定性系统之间建立契约。例如，像 `getWeather("NYC")` 这样的函数调用，每次被调用时，都会以完全相同的方式获取纽约市的天气。

工具是一种新型软件，体现的是确定性系统与非确定性 Agent 之间的契约。当用户问“今天我应该带伞吗？”时，Agent 可能调用天气工具，也可能凭一般知识回答，甚至可能先追问用户所在的位置。有时，Agent 可能产生幻觉，甚至无法理解该如何使用某个工具。

这意味着，为 Agent 编写软件时，我们需要从根本上重新思考方法：不能再像为其他开发者或系统编写函数和 API 那样编写工具和 MCP 服务器，而应当面向 Agent 来设计它们。

我们的目标是扩大 Agent 能够有效解决任务的范围，让它们通过工具采取多种可行策略，完成广泛的任务。幸运的是，根据我们的经验，对 Agent 而言最“顺手”的工具，往往也出乎意料地容易被人类直观理解。

如何编写工具

本节介绍如何与 Agent 协作，编写并改进供它们使用的工具。首先快速搭建工具原型，并在本地测试。接着，运行全面的评测，衡量后续修改的影响。通过与 Agent 协作，你可以反复评测和改进工具，直到 Agent 在现实任务中取得良好表现。

构建原型

如果不亲自动手，很难预先判断哪些工具会让 Agent 用起来顺手，哪些不会。首先快速搭建工具原型。如果你使用 Claude Code 编写工具（甚至可能一次生成），为 Claude 提供工具所依赖的软件库、API 或 SDK 的文档会很有帮助，其中也可能包括 MCP SDK。适合 LLM 阅读的文档通常可以在官方文档网站的扁平化

`llms.txt` 文件找到（这是我们 API 的文档）。

将工具封装为本地 MCP 服务器或桌面扩展（DXT），就可以在 Claude Code 或 Claude 桌面应用中连接并测试工具。

要将本地 MCP 服务器连接到 Claude Code，请运行 `claude mcp add <name> <command> [args...]`。

要将本地 MCP 服务器或 DXT 连接到 Claude 桌面应用，请分别前往 `Settings > Developer` 或 `Settings > Extensions`。

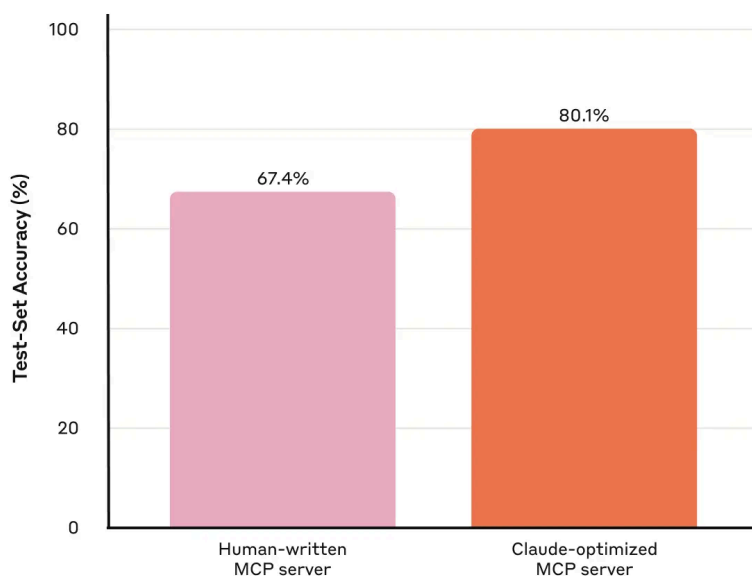
也可以将工具直接传入 Anthropic API 调用，以编程方式进行测试。

亲自测试工具，找出不顺畅之处。收集用户反馈，逐步形成对预期使用场景和提示词的直觉，了解你的工具应当支持哪些需求。

运行评测

接下来，需要通过评测衡量 Claude 使用工具的效果。首先，基于真实用途生成大量评测任务。我们建议与 Agent 协作，让它帮助分析结果，并判断如何改进工具。完整流程可参见我们的工具评测实践指南。

Slack tools



Held-out test set performance of our internal Slack tools

生成评测任务

有了早期原型，Claude Code 就可以快速探索你的工具，并创建数十组提示词与响应配对。提示词应来自真实使用场景，并基于贴近实际的数据源和服务，例如内部知识库和微服务。我们建议避免过于简单或流于表面的“沙箱”环境，因为它们的复杂度不足以对工具进行充分的压力测试。好的评测任务可能需要多次工具调用，甚至数十次。

以下是一些好的任务示例：

- 安排下周与 Jane 开会，讨论我们最新的 Acme Corp 项目。附上上一次项目规划会议的记录，并预订一间会议室。
- 客户 ID 为 9182 的客户反映，一次购买尝试被扣款三次。找出所有相关日志条目，并判断是否有其他客户受到同一问题的影响。
- 客户 Sarah Chen 刚刚提交了取消服务的请求。请准备一份挽留优惠方案。确定：(1) 客户为什么要离开；(2) 什么挽留优惠最有吸引力；(3) 提出优惠方案前，我们应了解哪些风险因素。

以下则是一些较弱的任务：

- 安排下周与 jane@acme.corp 开会。
- 在支付日志中搜索 `purchase_complete` 和 `customer_id=9182`。
- 查找客户 ID 为 45892 的取消请求。

每条评测提示词都应配有可验证的响应或结果。验证器可以很简单，例如将标准答案与采样生成的响应进行精确字符串比较；也可以更复杂，例如让 Claude 评判响应。应避免过于严格的验证器，以免仅因格式、标点或合理的替代表述等无关差异，就将正确响应判为错误。

对于每组提示词与响应，你还可以选择指定预期 Agent 为解决任务而调用的工具，从而在评测中衡量 Agent 是否成功理解了每个工具的用途。不过，正确完成任务可能有多条有效路径，因此应尽量避免对策略规定得过细，或对某种策略过拟合。

运行评测

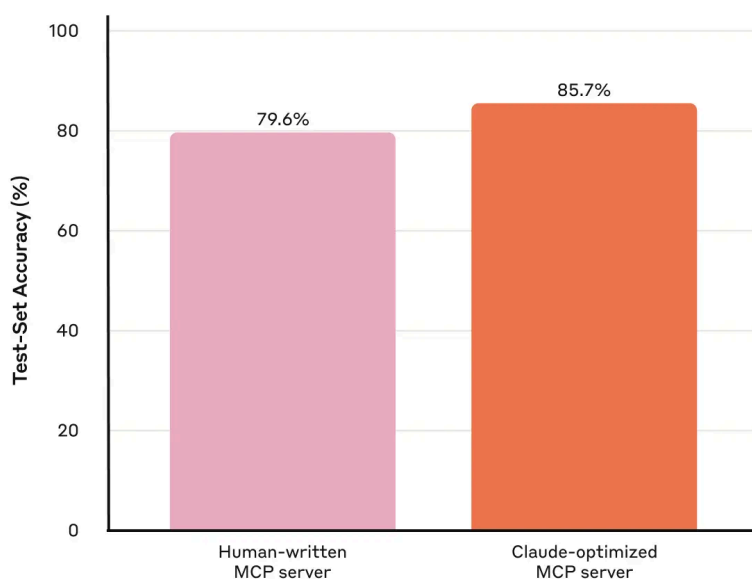
我们建议直接调用 LLM API，以编程方式运行评测。采用简单的 Agent 循环，即用 `while` 循环封装交替进行的 LLM API 调用与工具调用，每个评测任务对应一个循环。应向每个评测 Agent 提供一条任务提示词和你的工具。

在评测 Agent 的系统提示词中，我们建议要求 Agent 不仅输出用于验证的结构化响应块，还输出推理块和反馈块。要求 Agent 在工具调用块和响应块之前输出这些内容，可能会通过触发思维链（CoT）行为，提高 LLM 实际发挥出的智能水平。

如果使用 Claude 运行评测，可以开启交错思考，直接获得类似的现成功能。这有助于探究 Agent 为什么调用或不调用某些工具，并找出工具描述和规格说明中需要改进的具体部分。

除了整体准确率，我们还建议收集其他指标，例如单次工具调用和任务的总运行时长、工具调用总次数、token 总消耗量，以及工具错误。跟踪工具调用有助于发现 Agent 经常采用的工作流程，并识别可以整合工具功能的机会。

Asana tools



Held-out test set performance of our internal Asana tools

分析结果

Agent 可以成为有用的伙伴，帮助发现问题并提供反馈，涉及的范围包括相互矛盾的工具描述、低效的工具实现，以及令人困惑的工具 schema。不过，请记住：Agent 在反馈和响应中遗漏的内容，往往可能比它写出的内容更重要。LLM 并不总能如实表达其实际思考。

观察 Agent 在哪里卡住或感到困惑。通读评测 Agent 的推理和反馈（或思维链），找出不顺畅之处。检查原始交互记录，包括工具调用和工具响应，捕捉 Agent 未在思维链中明确描述的行为。要读出字里行间的信息；记住，评测 Agent 并不一定知道正确答案和正确策略。

分析工具调用指标。大量冗余工具调用可能说明应适当调整分页或 token 上限参数；大量因参数无效产生的工具错误，可能说明工具需要更清晰的描述或更好的示例。我们推出 Claude 的网页搜索工具时，发现 Claude 会不必要地在工具的 `query` 参数中追加 `2025`，导致搜索结果产生偏差，降低表现。我们通过改进工具描述，将 Claude 引导到了正确方向。

与 Agent 协作

你甚至可以让 Agent 分析结果，并替你改进工具。只需将各个评测 Agent 的交互记录拼接起来，粘贴到 Claude Code 中即可。Claude 擅长分析交互记录，并同时重构大量工具，例如在引入新修改时，确保工具实现与描述保持一致。

事实上，本文大部分建议都来自我们使用 Claude Code 反复优化内部工具实现的过程。我们的评测基于内部工作空间创建，复现了内部工作流程的复杂性，其中包含真实的项目、文档和消息。

我们依靠预留测试集，确保没有对“训练”评测过拟合。这些测试集表明，即使工具已经由“专家”实现，我们仍然能够在其基础上进一步提升表现，无论这些工具是由研究人员手动编写，还是由 Claude 自己生成。

下一节将分享我们从这一过程中学到的一些经验。

编写有效工具的原则

本节将我们的经验提炼为几条指导原则，帮助编写有效的工具。

为 Agent 选择合适的工具

工具更多，并不总能带来更好的结果。我们观察到的一个常见错误是，工具只是对现有软件功能或 API 端点做了一层封装，却没有考虑这些工具是否适合 Agent。原因在于，Agent 与传统软件有着不同的“可供性（交互能力）”，也就是说，它们感知自己能够通过这些工具采取哪些行动的方式不同。

LLM Agent 的“上下文”有限，也就是它们一次能够处理的信息量有限，而计算机内存则便宜且充裕。以在通讯录中查找联系人为例，传统软件程序可以高效地存储联系人列表，逐个处理联系人，检查完一个再继续下一个。

但如果 LLM Agent 使用的工具返回了全部联系人，它就必须逐个 token 阅读每个联系人的信息，把有限的上下文空间浪费在无关信息上。想象一下，为了在通讯录中找到某个联系人，你从头到尾逐页阅读，也就是采用暴力搜索。对 Agent 和人类而言，更好、更自然的做法都是先跳到相关页面，例如按字母顺序定位。

我们建议先针对特定且影响较大的工作流程，精心设计少量工具，使其与你的评测任务相匹配，再在此基础上扩展。以通讯录为例，可以选择实现 `search_contacts` 或 `message_contact` 工具，而不是 `list_contacts` 工具。

工具可以整合功能，在内部处理多个独立操作或 API 调用。例如，工具可以在响应中补充相关元数据，或用一次工具调用处理经常串联执行的多步骤任务。

以下是一些例子：

- 与其分别实现 `list_users`、`list_events` 和 `create_event` 工具，不如考虑实现一个 `schedule_event` 工具，查找空闲时间并安排活动。
- 与其实实现 `read_logs` 工具，不如考虑实现一个 `search_logs` 工具，仅返回相关日志行及其周围的一些上下文。
- 与其分别实现 `get_customer_by_id`、`list_transactions` 和 `list_notes` 工具，不如实现一个 `get_customer_context` 工具，一次性汇总某位客户近期的所有相关信息。

确保构建的每个工具都有明确且独立的用途。工具应让 Agent 能够像拥有相同底层资源的人类一样拆解和解决任务，同时减少原本会被中间输出占用的上下文。

工具过多或功能重叠，也可能干扰 Agent 采用高效策略。认真、有选择地规划要构建哪些工具、不要构建哪些工具，确实能够带来显著回报。

为工具设置命名空间

你的 AI Agent 可能会接入数十个 MCP 服务器和数百种不同工具，其中包括其他开发者提供的工具。当工具功能重叠或用途模糊时，Agent 可能会困惑，不知道该使用哪一个。

命名空间，即通过共同前缀对相关工具进行分组，有助于划定大量工具之间的边界；MCP 客户端有时会默认这样做。例如，按服务设置工具命名空间，如 `asana_search`、`jira_search`，以及按资源设置命名空间，如 `asana_projects_search`、`asana_users_search`，可以帮助 Agent 在合适的时机选择正确的工具。

我们发现，在采用前缀还是后缀来设置命名空间之间作出选择，会对工具使用评测产生不可忽视的影响。影响因 LLM 而异，因此我们鼓励你根据自己的评测选择命名方案。

Agent 可能调用错误的工具，给正确的工具传入错误的参数，调用的工具过少，或错误地处理工具响应。通过有选择地实现那些名称能反映任务自然拆分方式的工具，你既能减少加载到 Agent 上下文中的工具及工具描述数量，又能把原本由 Agent 在上下文中完成的计算转移到工具调用本身。这会降低 Agent 整体犯错的风险。

让工具返回有意义的上下文

同样，工具实现应注意只向 Agent 返回高信息量的内容。与灵活性相比，应优先考虑内容与当前上下文的相关性，并避免使用底层技术标识符，例如 `uuid`、`256px_image_url`、`mime_type`。相比之下，`name`、`image_url` 和 `file_type` 等字段，更有可能直接为 Agent 后续的行动和响应提供依据。

与晦涩的标识符相比，Agent 通常也更擅长处理自然语言名称、术语或标识符。我们发现，仅仅把任意字母数字组合的 UUID 转换成语义更明确、更容易解释的语言，甚至只是改成从 0 开始编号的 ID 方案，就能通过减少幻觉，显著提高 Claude 在检索任务中的精确率。

在某些情况下，Agent 可能需要灵活处理同时含有自然语言和技术标识符的输出，哪怕只是为了发起后续工具调用，例如 `search_user(name='jane')` → `send_message(id=12345)`。你可以在工具中提供一个简单的 `response_format` 枚举参数，同时支持两种形式，让 Agent 控制工具返回 `"concise"`（简洁）还是 `"detailed"`（详细）响应，见下图。

还可以增加更多格式，以获得更大的灵活性，类似于 GraphQL 中可以精确选择想接收哪些信息。下面是一个用于控制工具响应详细程度的 `ResponseFormat` 枚举示例：

```
enum ResponseFormat {
    DETAILED = "detailed",
    CONCISE = "concise"
}
```

下面是一个详细工具响应的示例（206 个 token）：

```
• I'll search slack for recent bug reports and use the detailed format to see which channel IDs and threads to investigate further.

• slack - search (MCP)(query: "bug", sort: "timestamp", sortDir: "desc", limit: 100, responseFormat: "detailed")
  L Search results for: "bug"

  === Result 1 of 89 ===
  Channel: #dev (C1234567890)
  From: @jane.doe (U123456789)
  Time: 2024-01-15 10:30:45 UTC
  TS: 1705316445.123456
  Text: Found a critical bug in the login flow.

  === Result 2 of 89 ===
  Channel: DM with @john.smith
  From: @john.smith (U987654321)
  Time: 2024-01-14 15:22:18 UTC
  TS: 1705247738.234567
  Text: The bug report for issue #123 is ready for review
  Files: bug-report-123.pdf
  ...
```

下面是一个简洁工具响应的示例（72 个 token）：

```
• I'll search slack for recent bug reports and use the concise format to read as many messages as possible.

• slack - search (MCP)(query: "bug", sort: "timestamp", sortDir: "desc", limit: 100, responseFormat: "concise")
  L Search: "bug" (89 results)

  1. #dev - @jane.doe: Found a critical bug in the login flow. [Jan 15]
  2. DM - @john.smith: The bug report for issue #123 is ready for review [Jan 14]
  ...
```

Slack threads and thread replies are identified by unique `thread_ts` which are required to fetch thread replies. `thread_ts` and other IDs (`channel_id`, `user_id`) can be retrieved from a `"detailed"` tool response to enable further tool calls that require these. `"concise"` tool responses return only thread content and exclude IDs. In this example, we use $\sim\frac{1}{3}$ of the tokens with `"concise"` tool responses.

甚至工具响应的结构，例如 XML、JSON 或 Markdown，也会影响评测表现；不存在适用于所有情况的解决方案。这是因为 LLM 通过预测下一个 token 进行训练，通常在格式与训练数据相符时表现更好。最佳响应结构会随任务和 Agent 的不同而存在很大差异。我们鼓励你根据自己的评测选择最佳响应结构。

优化工具响应，提高 token 使用效率

优化上下文质量很重要。但优化工具响应返回给 Agent 的上下文数量，同样重要。

对于任何可能占用大量上下文的工具响应，我们建议组合使用分页、范围选择、过滤和 / 或截断，并设置合理的默认参数值。在 Claude Code 中，我们默认将工具响应限制在 25,000 个 token 以内。我们预计 Agent 的有效上下文长度会随时间增长，但对高效使用上下文的工具的需求仍将持续存在。

如果选择截断响应，一定要提供有帮助的指引。可以直接鼓励 Agent 采用更节省 token 的策略，例如在知识检索任务中，多次进行小范围、有针对性的搜索，而不是只进行一次宽泛搜索。同样，如果工具调用报错，例如输入验证失败，可以通过提示词工程优化错误响应，清楚传达具体且可执行的改进方法，而不是返回不透明的错误码或堆栈追踪。

下面是一个工具响应被截断的示例：

```
● I'll find all of your transactions on Stripe and provide a summary for you.

● stripe - transactions_search (MCP)(limit: 5000, responseFormat: "concise")
  L ## Transaction Search Results

    Found **2,847 transactions** matching your query.

    The results are truncated. Showing first 3 results:

    | Date | Description | Amount | Category |
    |-----|-----|-----|-----|
    | 2024-01-15 | Payment from Acme Corp | +$5,200.00 | Revenue |
    | 2024-01-14 | Payment from TechStart | +$3,100.00 | Revenue |
    | 2024-01-13 | Payment from Cloud Co. | +$3,100.00 | Revenue |

    **Summary of all 2,847 results:**
    - Total Revenue: $458,291.00
    - Date Range: Jan 1 - Jan 15, 2024

    ## To refine these results, you can:

    - **Search for specific vendors:** Use `transactions_search(payee: "Acme Corp")` to see only Acme Corp charges
    - **Filter by amount range:** Use `transactions_search(minAmount: 1000, maxAmount: 5000)`
    - **Get next page:** Use `transactions_search(query: <query>, page: 2)`
```

下面是一个没有帮助的错误响应示例：

```
● Sure. I'll fetch John's contact information from his profile.

● asana - user_info (MCP)(userId: "john.doe@acme.corp")
  L {
    "error": {
      "code": "RESOURCE_NOT_FOUND",
      "status": 422,
      "message": "Invalid value",
      "details": {
        "field": "userId",
        "value": "john.doe@acme.corp",
        "type": "invalid_value"
      }
    }
  }
}
```

下面是一个有帮助的错误响应示例：

```
• Sure. I'll fetch John's contact information from his profile.

• asana - user_info (MCP)(userId: "john.doe@acme.corp")
  L Tool Response:

    # Resource Not Found: Invalid `userId`

    ## Error Summary
    Your request to `/api/user/info` failed because the `userId` `john.doe@acme.corp` does not exist or is in
    the wrong format.

    ## Valid User IDs
    Examples:
    - `1928298149291729`
    - `9381719375914731`

    ## Resolving a User ID
    - Call user_search()
```

Tool truncation and error responses can steer agents towards more token-efficient tool-use behaviors (using filters or pagination) or give examples of correctly formatted tool inputs.

对工具描述进行提示词工程优化

现在介绍改进工具最有效的方法之一：对工具描述和规格说明进行提示词工程优化。由于这些内容会加载到 Agent 的上下文中，它们可以共同引导 Agent 形成有效的工具调用行为。

编写工具描述和规格说明时，可以设想你会如何向团队的一位新同事介绍工具。考虑那些你可能默认对方已经知道的背景，例如专用查询格式、小众术语的定义、底层资源之间的关系，并把它们明确写出来。清楚描述预期的输入和输出，并用严格的数据模型加以约束，以避免歧义。尤其是输入参数的名称，应当明确无误：例如，不要将参数命名为 `user`，可以改用 `user_id`。

借助评测，你可以更有把握地衡量提示词工程的影响。即便只是微调工具描述，也可能带来大幅提升。我们对工具描述进行精准调整后，Claude Sonnet 3.5 在 SWE-bench Verified 评测中取得了当时最先进的表现，错误率大幅降低，任务完成情况得到改善。

其他工具定义的最佳实践可参见我们的开发者指南。如果你正在为 Claude 构建工具，我们还建议了解工具如何动态加载到 Claude 的系统提示词中。最后，如果你正在为 MCP 服务器编写工具，工具注解有助于说明哪些工具需要访问外部开放环境，或会造成破坏性修改。

展望未来

要为 Agent 构建有效工具，我们需要调整软件开发实践，将重心从可预测的确定性模式转向非确定性模式。

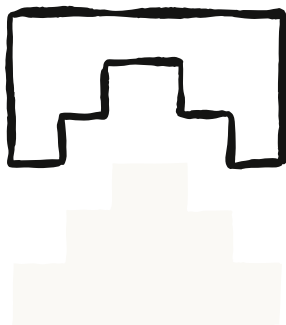
通过本文描述的迭代式、评测驱动的过程，我们发现了让工具成功发挥作用的一些共同规律：有效的工具经过有意识且清晰的定义，审慎使用 Agent 的上下文，能够组合成多种工作流程，并让 Agent 以直观的方式解决现实任务。

未来，我们预计 Agent 与世界交互的具体机制还会持续演进，从 MCP 协议的更新，到底层 LLM 本身的升级，都是如此。通过系统化、评测驱动的方法改进 Agent 的工具，我们可以确保：随着 Agent 能力增强，它们使用的工具也会同步演进。

致谢

本文由 Ken Aizawa 撰写，研究团队（Barry Zhang、Zachary Witten、Daniel Jiang、Sami Al-Sheikh、Matt Bell、Maggie Vo）、MCP 团队（Theodora Chu、John Welsh、David Soria Parra、Adam Jones）、产品工程团队（Santiago Seira）、市场团队（Molly Vorwerck）、设计团队（Drew Roper）和应用 AI 团队（Christian Ryan、Alexander Bricken）的同事作出了宝贵贡献。

¹本文讨论的是训练底层 LLM 本身之外的方法。



想进一步了解？

[探索课程](#)